National College of Ireland

Machine Learning-Based Cost-Performance Optimization for Multi-Cloud Serverless Orchestration

MSc Research Project
Programme Name

Forename Surname
Student ID: XXX

School of Computing
National College of Ireland

Supervisor: XXX

National College of Ireland

MSc Project Submission Sheet

School of Computing

Student Name: …….
……
……

Student ID: ……
……..……

Programme: …… **Year:** ……
…. .

Module: ……
……..……

Supervisor: ……
……..……

Submission Due Date: ……
……..……

Project Title: ……
……..……

Word Count: …… **Page Count**……..……..

I hereby certify that the information contained in this (my submission) is information pertaining to research I conducted for this project. All information other than my own contribution will be fully referenced and listed in the relevant bibliography section at the rear of the project.
ALL internet material must be referenced in the bibliography section. Students are required to use the Referencing Standard specified in the report template. To use other author's written or electronic work is illegal (plagiarism) and may result in disciplinary action.

Signature: ……

Date: ……
……

PLEASE READ THE FOLLOWING INSTRUCTIONS AND CHECKLIST

Attach a completed copy of this sheet to each project (including multiple copies)	□
Attach a Moodle submission receipt of the online project submission, to each project (including multiple copies).	□
You must ensure that you retain a HARD COPY of the project, both for your own reference and in case a project is lost or mislaid. It is not sufficient to keep a copy on computer.	□

Assignments that are submitted to the Programme Coordinator Office must be placed into the assignment box located outside the office.

Office Use Only	
Signature:	
Date:	
Penalty Applied (if applicable):	

Machine Learning-Based Cost-Performance Optimization for Multi-Cloud Serverless Orchestration

Forename Surname
Student ID

Abstract

Multi-cloud serverless orchestration is subject to vendor lock-in and the sustainability issue that necessitates solving the critical challenges such as optimization of resource allocation, cost management, and performance assurance. Existing orchestration frameworks based on the simple heuristics are clearly inadequate for the workload pattern's dynamic nature and the changing cloud provider conditions. This study presents a novel approach by deep reinforcement learning (DRL)-based orchestration for multi-objective optimization in the serverless environments of multi-clouds. The framework proceeds by hierarchical decision-making utilizing the Deep Q-Network (DQN) for cloud selection strategy, Proximal Policy Optimization (PPO) for tactical function placement, and LSTM-based predictive models for operational resource allocation. Moreover, a vendor-neutral abstraction layer, which helps in mitigating lock-in risks, and a carbon-aware scheduling algorithm, which incorporates sustainability into the optimization process, are used in the framework. The SLA compliance is prioritized by the asymmetric loss function, which assigns penalties to under-provisioning while over-provisioning is less penalized. The framework is supposed to achieve lower costs while being performance-friendly based on the recent DRL-based serverless optimization. This, together with a certain percentage of carbon footprint decrease, would be the proof of the efficacy of the framework. This research is a step forward in the field because it is the first to simultaneously address cost, performance, and environmental sustainability in multi-cloud serverless orchestration.

1 Introduction

1.1 Background and Motivation

Serverless computing has been one of the disruptive transformative technologies in the deployment of cloud applications, which has changed the ways in which organizations design, deploy, and manage computing loads. In contrast to infrastructure models that require explicit server provisioning and management, serverless architectures allow the programmers to focus exclusively on the application code while resource allocation is automatically done by the cloud providers, infrastructure scaling, and maintenance (Castro et al., 2019). The Function-as-a-Service (FaaS) system model is event-driven and pay-as-you-go, where applications are composed of stateless and ephemeral functions invoked by events, which are executed in milliseconds to seconds and terminated later (Jonas et al., 2019). The promotional growth of the serverless computing market has been revolutionary with a forecast indicating the market would touch $22 billion by 2025, with the prior being the propeller of low operational costs,

auto elasticity, and cost benefits when compared to traditional virtual machine deployments (Alhosban et al., 2024).

The strategy of adopting multiple cloud vendors has become a trend, with around 76% of organizations using multiple providers for vendor lock-in risk avoidance, proper cost optimization through different pricing models, and geographic distribution for business continuity (Alhosban et al., 2024). Reports point to the fact that serverless architectures can help save as much as 42-70% of energy and drop costs by 30-60% when viewed as compared to the conventional virtual machine-based systems (Akour and Alenezi, 2025). Yet, running serverless applications across multiple cloud platforms leads to real and difficult problems. The providers are heterogeneous in terms of API structure, pricing scheme, performance statistics, service-level agreement, etc. Moreover, sustainability issues are getting more attention, which forces the organizations to reduce their carbon emissions with the help of regulations that the government puts on them, like the EU AI Act (Jiang et al., 2024). The combination of financial stress, performance demands, sustainability commitments, and regulatory compliance necessitates a smart orchestration mechanism that can navigate through the inherent multi-cloud serverless deployment trade-offs.

1.2 Problem Statement

The focus of multi-cloud serverless orchestration is to balance the optimal resource allocation among diverse cloud infrastructures with different pricing models, performance characteristics, and operational constraints. Current orchestration frameworks rely primarily on fixed policies or the simplest heuristics that cannot adjust to the varied load patterns or temporary changes in the cloud provider's environment (Chen et al., 2025). The non-deterministic serverless execution effect is mainly seen as cold start latencies, which vary based on runtime environments, function dependencies, and resource initialization, thus significantly affecting users' experience and application performance (Golec et al., 2024).

The current multi-cloud orchestration solutions are heavily focused on single-cloud support, which in turn leads to vendor lock-in risks with varying levels of dependency for the different service combinations (Alhosban et al., 2024). The lack of standardized APIs and interoperability protocols among cloud providers generates an annual financial loss in migration costs of about 15% to 40%. What's more, cross-cloud data transfer costs create substantial economic burdens because of egress charges and network latency penalties of 50-200 ms due to geographical distribution (Golec et al., 2024). The focus on sustainability issues, in turn, is adding pressure on organizations to achieve their goals at costs that are on par with previous years' performance, which necessitates the use of intelligent orchestration mechanisms capable of navigating the complex trade-offs inherent in multi-cloud serverless deployments.

1.3 Research Question

What identifiable benefits can be obtained in cost and performance efficiency, prediction accuracy of workloads, and latency reduction through multi-objective orchestration in multi-cloud serverless environments based on deep reinforcement learning, while also reducing the carbon footprint?

1.4 Problem Solution

This research thoroughly develops a DRL communication architecture that addresses multi-cloud serverless overhead through a hierarchical decision-making framework orchestrating strategic, tactical, and operational levels. In the strategic layer, a DQN agent selects the governing cloud provider by analyzing the historical cost records, efficiency metrics, and carbon footprint of each individual cloud. The tactical layer utilizes Proximal Policy Optimization (PPO) for optimal placement of functions across regions and availability zones, factoring in data locality, cold start probabilities, and inter-cloud communication costs. In this operational layer, LSTM-based predictive models are the backbone for the forecast of the real-time resource demand and dynamic allocation. The framework also contains an asymmetric loss function that, at the end of the day, penalizes under-provisioning more than over-provisioning in the real world, where SLA breaches have greater costs than just slight resources left over. A vendor-neutral abstraction layer translates high-level orchestration decisions into platform-specific API calls, thus ensuring compatibility for multiple platforms such as AWS Step Functions, Google Cloud Workflows, Azure Durable Functions, and OpenFaaS. The addition of carbon-aware scheduling algorithms in the framework makes it possible to include sustainability metrics together with traditional cost and performance motives, which will support eco-friendly cloud resource management while keeping similar performance levels.

1.5 Research Objectives

The main objectives of this research include,

1. The creation of a DRL-based orchestration framework that will be treating multi-cloud serverless resource allocation as a multi-objective optimization problem.
2. The construction and initiation of a multi-objective optimization algorithm that simultaneously optimizes the cost performance and real-time carbon footprint by incorporating sustainability into the cloud resource management decisions.
3. The development of a sophisticated cost prediction model involving LSTM networks and asymmetric loss functions that will achieve less prediction error compared to the traditional methods, leading to improved cost planning and proactive cloud optimization.
4. The establishment of a vendor-neutral abstraction layer that can quantify and mitigate vendor lock-in risks through the seamless orchestration of cross-cloud functions, thus decreasing migration complexity and allowing decision makers to optimize the cloud utilization without the fear of incurring prohibitive migration costs.
5. To ensure the success of the proposed framework through an empirical evaluation that is done on the real-world workloads from Azure and Google Cloud platforms.

1.6 Challenges and Limitations

The research acknowledges some minor challenges and limitations, such as theoretical constraints, which can affect the scope and outcomes of the proposed framework. The main challenge is the dynamic complexity of solving NP-hard multi-objective optimization problems in real-time, as this will require a careful emphasis on the trade-off between the solution quality

and the decision-making speed to be practical for the production environments. The training of DRL models on historical workload traces might bring in potential biases and thus limit the generalizing capability among different application areas, mainly among the new workload patterns that were not present in the training data. The variation in cloud provider APIs, pricing models, and service-level guarantees necessitates the ongoing maintenance of the abstraction layer to adapt to the provider-specific updates and new service offers. The calculation hazard in measuring the carbon footprint is that the hardware embodied carbon data is somewhat dependent on both the manufacturers and their estimations. This, in turn, may affect the integrity of the derived sustainability metrics. The capability of the framework to manage sudden extreme workload changes or cloud provider failure needs to be thoroughly tested in different failure scenarios.

2 Related Work

2.1 Multi-Cloud Serverless Orchestration

Calavaro et al. (2025) feature a survey study that revolves around deploying, challenges, frameworks, resource management, and applications of Function-as-a-Service (FaaS) in edge-to-cloud continuum environments. A systematic literature review is performed by the authors of the paper on key challenges categorized into several issues: system frameworks and infrastructures, resource and function management, sustainability concerns, security and privacy issues, and application domains. They mention two architectural methods for the cloud-centric headwind: first, to use lightweight isolation mechanisms with WebAssembly and unikernels to minimize the initialization time and memory footprint, and second, to utilize decentralized architecture to avoid the centralized scheduler bottleneck. The survey presents specific frameworks, such as Serverledge (with live migration capabilities), Colony (parallel processing workloads with automatic workflow conversion), TEMPOS (real-time scheduling), and GeoFaaS (vertical offloading with location-based routing). The survey asserts that most of the advances in these areas have been great, yet deploying FaaS efficiently still requires additional efforts to be done in the form of joint frameworks that take care of performance, energy efficiency, security, and programmability challenges together.

Zhao et al. (2022) introduce a multi-cloud library for the cross-serverless offerings along with an End Analysis system (EAS) that enables performance and cost comparison among public FaaS providers. The issue of concern is the incapacity to optimally use multiple FaaS providers for several workloads due to the closure of data egress fees, which further adds to the data gravity effects, locking the customers into hiring respective vendors. The design architecture introduces two proofs thus: a pipeline-friendly multi-cloud architecture integrating Cloudflare R2 for the zero-egress fee storage and a pipeline-unaware one. Both utilize image processing and machine learning training workloads across respective clouds: AWS Lambda, Google Cloud Functions, and Alibaba Function Compute. The evaluation metrics focus on the amount of time during which the billing lasted, 99th percentile latency, request fees, network fees, duration fees per million requests, and more. The EAS enables predicting costs finely and analyzing performance as well, while the multi-cloud library enables the portability of

workloads through common interfaces and SDK conversion capabilities, which constitute such a challenge as abstracting the mix of cloud resources.

Peri et al. (2024) introduce the Hybrid Cloud Scheduler (HCS) for the efficient orchestration of serverless batch-processing pipelines across private edges and public clouds entirely based on cost. The paper talks about three main focuses: the uneven function invocations that are significantly longer than expected on average, the limited resources such as the core cluster having no scalability options, and the urgency to fulfill the deadlines in a more cost-beneficial way. The Batch Driver (that accomplishes job submission, monitoring, and fault tolerance actions through consistent journaling in etcd) and the HCS Scheduler (whose key function is a two-level scheduling approach that determines policy and placement decisions) make up the two components of the system. By defining the batch jobs as the directed acyclic graphs (DAGs), the scheduling methodology shows that feed-forward execution is used, which benefits the immediate propagation of mini-batches to subsequent processing steps. The private edge cluster scheduler evaluates four placement policies: First-Fit, Best-Fit, Round Robin, and Worst-Fit, which are representative of different bin-packing heuristics. The experimental total cost of using OpenFaaS on Apache Mesos with Marathon orchestration dropped to 87% of cloud-only baseline costs with very high rates of SLO compliance.

Filinis et al. (2024) illustrate an intent-driven orchestration way of dealing with serverless applications in the computing continuum from edge to cloud infrastructure. The research aims at solving three tightly connected issues: easy resource abstraction for application developers, resource management of distributed multi-clusters being a single entity, and dynamic scaling with function chain dependencies. The solution is to reinstate the power grid with reinforcement learning-driven autoscaling and function placement with multiple-objective optimization. This presents different serverless functions within a chain that consequently lead to different latencies and computational requirements, making it necessary for smart placements to be decided. The RL-based autoscaling mechanism utilizes a deep Q-network (DQN) along with a very well-designed reward function, which optimizes resource utilization while keeping average latency below the service level agreement (SLA) thresholds by allowing minimum replica counts across functions. Using NASA server HTTP traces that show 1.52% of service being lost along with a slight amount of replica being over-provisioned, but on the other hand, achieving a lower 22% transformation cost and 24.2% less energy expenditure compared to First-Fit scheduling were the criteria for the framework evaluation. Thus, the intent specification allows the developers to declare high-level objectives such as performance, cost, and energy efficiency as well as properties such as autoscaling limits and concurrency.

2.2 ML and DRL for Resource Optimization

Chen et al. (2025) reveal function scheduling issues in serverless edge computing, with particular emphasis on different banking tasks, thus helping to address the unpredictability of pay-as-you-go pricing models that makes it difficult for firms to manage their financial budgets effectively. The issues discussed are related to the costs of three components: function switching cost, function running cost, and traffic routing cost. In the solution, two DRL paradigms are employed: DFaaS, which is based on DQN, and PFaaS using PPO. The Markov Decision Process (MDP) model lays out states in terms of the remaining hardware capacity,

hosted functions, and requests generated across nodes; actions specify which nodes to target and to generate containers; rewards are negative deployment costs. With the dataset of 141 days of invocations across 200 functions, the servers were trained to learn policies through an offline training approach on the Huawei serverless dataset traces, which included real topology data from 125 edge nodes deployed in the Melbourne CBD. Performance comparison with the Midaco ILP solver indicates that DFaaS and PFaaS are very close to the optimal with no more than 1.03 difference in the result. Their results were 122.35 and 124.00 average deployment costs in comparison to Midaco's 118.35, but decision-making time was reduced by five orders of magnitude from 24+ hours to 1.3-1.5 seconds. The acceptance rate gap was 60-100%; nonetheless, with learning-based approaches efficiently exploring solution spaces to accommodate more requests despite slightly higher costs.

Zhang et al. (2024) explore the region of secure and efficient resource allocation in serverless multi-cloud edge computing edges in which IoT and 6G applications are introduced along with the SARMTO (Secure Adaptive Resource Management and Task Optimization) model. Integrated within the action-constrained DRL model (AC-DQN), SARMTO is a resource management and task scheduling model that ensures system constraints are followed during the decision process, rather than simply punishing them, by mapping infeasible actions to large negative values. The formulation of the MDP covers security overhead for the encryption, decryption, and integrity verification. The time of security and energy overhead is explicitly modeled due to RSA and MD5 encryption, with the security mechanism dynamically adjusting the protection level according to task sensitivity, which is also modeled. Simulations that run SARMTO over varying conditions showed power savings of up to 40% annually, and energy efficiency improvements of 41.5% were achieved by state-of-the-art methods, including standard DQN, DQN with backpropagation neural networks, random offloading, local computation, and cloud layer offloading.

The MinionsRL project, being a unique framework for serverless distributed DRL training, was developed by (You et al., 2024) in order to resolve the resource waste issue observed in synchronous actor-learner architectures. The problem is that server-side DRL training with on-policy algorithms has idle actors during the periods of synchronization and policy updates, which leads to a significant resource waste and also increases the billing cost due to the inaccurate resource management on the server level, which takes place on a minute level, and latency is added to the startup. The framework points out that the dynamic change of their actor will be needed, learning what works best as the training progresses. MinionsRL is embedded with a DRL-based scheduler that employs PPO to find the best actor counts for each training round, thus balancing training quality and cost. The intelligent scheduler learns to give more actors during early exploration cases and to reduce them when the performance becomes stable, thus optimizing the actor budget that is specified as a total of actors in all rounds. The evaluation was held on Microsoft Azure Container Instances and utilized OpenAI Gym environments, which proved MinionsRL to save total training time by up to 52% and training cost by 86% when compared to Azure ML and IMPACT baselines. The breakdown for the framework's latency displayed that the actor/learner function startup is approximately 300 ms/1500 ms, respectively.

Yu et al. (2024) presented Stellaris, a groundbreaking asynchronous learning paradigm for serverless computing that is utilized for distributed DRL training. Eclipsing dynamic staleness and limited sample diversity to explore high exploration costs, this product, Stellaris, addresses the issues mentioned above. The problem with the asynchronous multi-learner serverless training is that there are three barriers, namely, dynamic learner orchestration, dynamic staleness, and unstable policy updates. Stellaris forwards a dual-layer architecture with experience-sharing features at the distributed level, in which multiple serverless environments are orchestrated in parallel, attached with DRL agents that are able to upload experience tuples for a global shared replay buffer hosted in the cloud, which is utilized for parallel exploration and data diversity enhancement. The population evolution search in collaboration with policy search describes multiple agents as a population that evolves across generations (M epochs each), selects the top individual based on fitness (the average rewards over the recent epochs), and builds an improved loss function that uses policy distance metrics to guide the rest of the individuals on the right track while continuing to explore via adaptive weighting. The proof of concept was executed on AWS EC2 testbeds and HPC clusters with 16 GPUs and 960 CPU cores, while benchmarks on MuJoCo and Atari games depicted a final reward training quality with a 2.2% hike and a 41% reduction of training cost alongside synchronous Ray RLlib.
Yao et al. (2023) present ES-DRL (Experience-Sharing DRL), a distributed function offloading method that deals with the serverless edge computing challenges by combining stateful and stateless execution models. The technique focuses on mitigating the effects of sample diversity and exploration costs with regard to the existing DRL-based task offloading, especially in the case of latency-sensitive IoT applications at the edge in the serverless FaaS environment. This work is the first of its kind to use offloading as a unit, taking into account the container startup time under the condition of stateless execution models, rather than performing coarse-grained task offloading, which is the usual method. The latency in IoT devices comprises computation time (CPU cycles divided by device capacity) plus local queue latency, while EFaaS latency includes transmission time (based on wireless channel rate considering SINR), cold vs. warm container, computation time (proportional to allocated container memory), and EFaaS queue latency. Simulation tests with 50 IoT devices included the implementation of the EFaaS supporting parallel function execution (2.5 GHz per core, 4 cores maximum per container), 20 MHz bandwidth, and functions requiring 100-1000 kB data, 0.6-2 GHz CPU cycles, and 128-512 MB memory exhibited that ES-DRL reduces average latency by around 17% compared to DDPG, increases function completion rates (fulfilling deadlines), and cuts down epochs of convergence from 65,000 (DDPG) to 38,000 (ES-DRL) for a total of 50 devices.

2.3 Carbon-Aware Cloud Computing

Jiang et al. (2024) developed ECOLIFE, which is the first carbon-aware serverless function scheduler co-optimizing both carbon footprint and performance by engaging intelligent exploitation of multi-generation hardware. The problem primarily considers the situation in which both embodied carbon and operational carbon result in an increased carbon footprint of data centers, while serverless computing necessitates especially the keep-alive carbon footprint to be accounted for together with execution emissions. ECOLIFE features extensions to Particle Swarm Optimization (PSO) with the application of novel methods including a

perception-response-based adaptability to serverless environments and varying carbon intensities, as well as a function warm pool mechanism that intelligently adjusts the function keep-alive time and location in response to the memory requirement of multi-generation hardware. Moreover, the scheduler solves both keep-alive locations (old vs. new hardware) and keep-alive periods for all invoked functions to minimize overall carbon footprint and service time. The evaluation used Microsoft Azure production serverless traces from SeBS benchmark functions (video processing, graph search, and DNA visualization) with three hardware pairs and indicated that ECOLIFE performs consistently within 7.7% and 5.5% of the theoretically optimal ORACLE solution in service time and carbon footprint, respectively. In their quantitative analysis, the authors disclosed that the keep-alive carbon footprint accounts for nearly 18-52% of the total function carbon footprint depending on the keep-alive duration (2-10 minutes), which emphasizes the importance of keep-alive period optimization.

Arys et al. (2025) put forward a method for scheduling serverless workloads in heterogeneous clusters in a software energy-efficient manner. The solution is built up by three main parts: offline profiling of functions for power consumption and performance with PowerAPI toolchain, creating performance/energy affinity models with function colocation impacts through offline profiling with different mixes of functions, and an energy-aware constraint programming scheduler that allocates CPU cores to match the needed throughput while minimizing the expected energy consumption. The testing equipment used is a dual-socket Intel Xeon Gold 5318Y server and an Ampere Altra Max ARM server, both having 256GB RAM and connected power distribution units to accurately measure the electricity consumption. Energy-aware (EA) minimizing the total projected energy over the functions is one of the two objective functions being compared with the cores-aware (CA) baseline, which minimizes allocated cores without consideration of the energy consumed. The evaluation was done using 22 representative serverless functions from the SeBS benchmark and new workloads, and it showed that energy-aware scheduling lowers energy requirements by up to 15.2% in settings where both servers are required to be involved in a consolidated operation to achieve the target throughput.

Chadha et al. (2023) present GreenCourier—a unique scheduling framework that can provide runtime scheduling of serverless functions across octal clusters according to their carbon efficiencies, which enables carbon delivery from performing functions. The issue arises from the fact that the commercial FaaS platform auto-schedules geographic regions for function deployment, without the knowledge of fluctuating carbon intensities across the regions and the time. GreenCourier extends Kubernetes with Knative as the serverless platform and Liqo for establishing geographically distributed multi-cluster topologies, enabling seamless pod scheduling across clusters. Carbon-aware scheduling policy is a plugin-based approach that executes a scoring plugin for the Kubernetes Scheduling Framework that, during the scoring run, after filtering, gets the current carbon scores from the metrics server for each eligible node's region, caches them locally for 5 minutes to reduce overhead, normalizes scores between 0 and 100, and selects the node with the lowest carbon score for pod assignment. Evaluation on Google Kubernetes Engine with production serverless traces revealed that GreenCourier reduces per-function invocation carbon emissions on average by 13.25% compared to the default Kubernetes scheduling (8.7% improvement) and geography-aware baseline (17.8%

improvement). In performance characterization the average geometric mean slowdown was 10.26% and 16.24% compared to default and geography-aware strategies, respectively, due to the selection of carbon-efficient but geographically distant regions.

Sharma and Fuerst (2024) developed the first energy and carbon metrology framework for FaaS functions, providing accurate full-system carbon footprints, including shared hardware and software emissions. The solution employs statistical disaggregation, leveraging FaaS workload locality properties to develop a simple practical online approach combining direct hardware measurements and model-based estimation. The energy profiling methodology uses a Kalman filter-inspired approach, providing online energy estimates without offline pre-training, enabling deployment in diverse heterogeneous cloud environments by adapting to changing workload dynamics. Fair energy attribution methods estimate fair-share contributions to shared software/hardware energy consumption using the Shapley value framework, providing theoretically grounded carbon footprint metrics incorporating both operational emissions (energy consumption multiplied by carbon intensity) and embodied emissions. Validation methodology introduces a rigorous marginal energy-based approach using workload trace replay, measuring server-level power after removing certain invocations to compute ground truth for the function's in-situ energy consumption. Evaluation across three hardware platforms using diverse FaaS workloads demonstrated energy footprints achieve >99% accuracy against marginal energy ground truth. Quantitative analysis reveals FaaS control plane contributes significant energy overhead (15-40% depending on function characteristics), validating the importance of including shared resource attribution.

2.4 Research Gap Analysis

Despite the great advances in the multi-cloud serverless orchestration landscape, the literature reviewed in the past merely dwelled on cost and performance aspects optimization, not even in terms of environmental sustainability. Despite demonstrated cost reduction of 20-30 percent through intelligent resource allocation by DRL methodologies, including that by Chen et al. (2025), carbon footprint is not taken into account in their objectives of optimization. Emission-aware planning systems such as EcoLife (Jiang et al., 2024) and GreenCourier also reduce carbon emissions by 13-18 percent, although they do not employ the full cost-performance optimization approaches. This fragmented methodology renders the possibility of a single framework to respond to the cost-effectiveness, performance, and sustainability measures concurrently in multi-cloud serverless orchestration.

The proposed study addresses this gap by applying a novel hierarchical DRL model integrating multi-objective optimization in the three domains, such as environmental impact, operational performance, and economic efficiency. The hypothesis is that the combination strategy of DQN to select a strategic cloud provider, PPO to place tactical functions, and LSTM-based predictive models to allocate operational resources, combined with carbon-aware scheduling algorithms and a vendor-neutral abstraction layer, will produce better results in all three of the objectives simultaneously. Standardized measures of evaluation will include a percentage of cost savings, average latency reduction, carbon footprint reduction, SLA compliance rates, and accuracy of prediction used as standardized evaluation metrics, which we will compare the framework to (Chen et al., 2025) in terms of cost-performance metrics, EcoLife in terms of

carbon optimization, and (Zhang et al., 2024) in terms of multi-cloud resource orchestration. This comparative assessment tool will ensure that the effectiveness of the proposed solution in filling the research gap is properly checked.

3 Research Methodology

This study embraces a normative experimental approach that integrates DRL with multi-objective optimization to create a smart orchestration framework for deploying multi-cloud serverless systems. The approach is structured and entails the preparation of data, the development of the models, deployment integration, and overall evaluation, which are aimed at addressing complexities of resource allocation in heterogeneous clouds.

3.1 Research Framework Overview

The proposed study adopts a hierarchical decision-making architecture that works at three different, yet interrelated, levels to control the complexity of multi-cloud serverless orchestration. At the strategic tier, a DQN agent uses past trends to make high-level selection decisions in the cloud provider based on long-term cost trends, performance metrics, and carbon intensities of various cloud providers. The tactical level executes PPO algorithms to decide the best position of functions in geographical regions and availability zones, considering such important aspects as data locality needs, cold start probability distributions, and inter-cloud communication latency penalties. The operational level combines Long Short-Term Memory (LSTM) networks to predict the workload in real-time and dynamically allocate resources to the framework to address the changing demand patterns in a dynamic manner. The multi-layered approach makes sure that the decisions at each level would be maximized in their individual time scales and can be coherent with the goals of the overall orchestration, serving as a holistic solution with immediate operational needs and long-term strategic costs and sustainability.

3.2 Research Process

The study adheres to a four-stage conceptual framework adopted in Figure 1, which involves preparing and formulating the data for model development and validation. Phase one is related to the overall preparation and preprocessing of data, during which historical workload traces of the Azure Functions Dataset 2021[1] are gathered, pre-cleaned, and divided into training, validation, and testing subsets with corresponding time cuts to avoid data leakage. The phase will involve exploratory data analysis to determine the workload trends, statistical representation of the execution times, profile of memory consumption, and feature engineering to derive the predictors that the machine learning models need. Phase two involves a cycle of model development where an initial step involves offline training of the DQN agent by experience replay mechanisms and fixed interval updates to the target network so that the learning process is stabilized. The PPO algorithm is highly optimized on hyperparameters by grid search exploring learning rates, clipping parameters and entropy coefficients to find

[1] https://github.com/Azure/AzurePublicDataset

optimal combinations. The LSTM predictor is trained with special consideration to temporal dependencies and checked in cases where concept drift occurs to guarantee good performance in changing workload trends. Phase three integrates the deployment and integration plan, containerizing all the elements with the help of Docker and managing them with Kubernetes clusters that are deployed on various cloud providers. The stage involves the creation of a vendor-neutral abstraction layer to convert high-level orchestration choices to platform-specific API calls to AWS Step Functions and Azure Durable Functions. Phase four performs full assessment and verification by controlled experiments that perform repeated comparisons of the suggested framework with baseline against approaches.

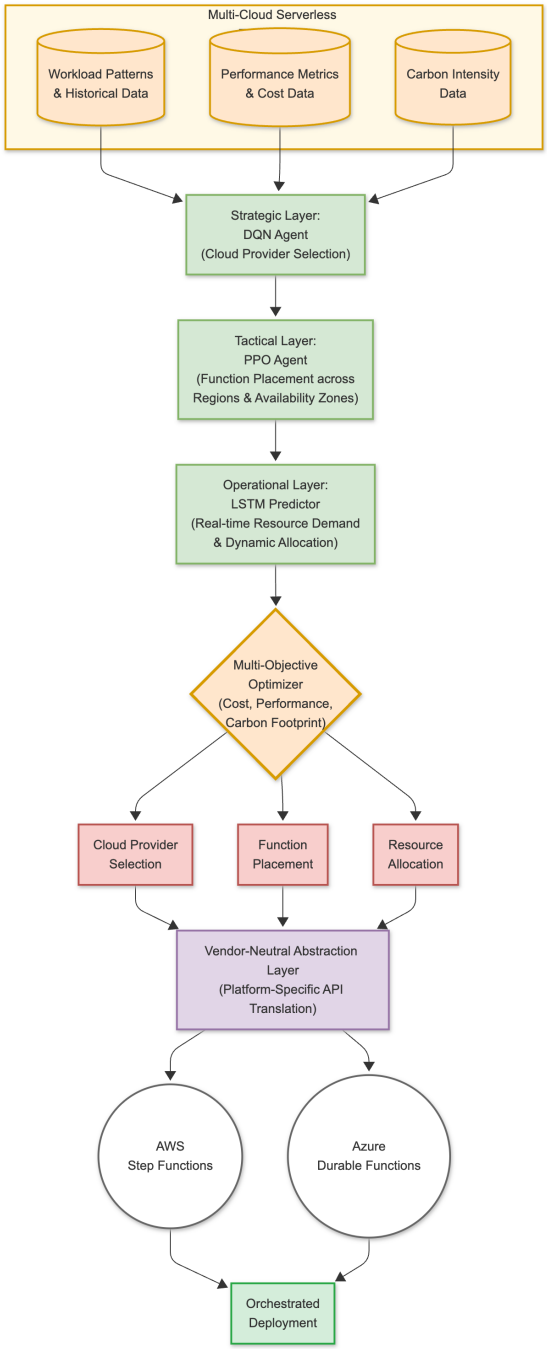


Figure 1: Proposed Research Framework

3.3 Dataset and Tools

The Azure Functions Dataset 2021 is the main dataset, which is the source of the more than 14 days of invocation traces from production workloads that have been given by this thing. The dataset contains details about the various types of servers used in the serverless architecture and the characteristics of the real-world serverless model. Along with this, Google Cluster Workload Traces 2019[2] will also be analyzed, which will add the multi-cluster workload dataset, and with that, the cross-cloud performance comparison can be done, and the framework can be verified for its generality in different ecosystems. Serverless Benchmarking Suite[3] is the research framework that is responsible for standardized benchmarking and synthetic workload generation, which provides language-agnostic benchmarks across multiple cloud platforms with automated deployment, execution, and metric collection capabilities for AWS Lambda, Azure Functions, Google Cloud Functions, and OpenWhisk. ML pipeline is realized in DRL with the use of PyTorch 2.0. The application is deployed as a container, which implies that we use Docker and Kubernetes for orchestration. This ensures that our infrastructure is heterogeneous but can be used to run the same application in a reproducible manner. The cost monitoring aspect is achieved with the help of the open-source framework, which collects the metrics in real-time. The carbon footprint measurements are also done by applying data from Electricity Maps[4] for the regional grids' carbon intensity, which is then added to the quantification of carbon emissions, both operational and embodied, from serverless function executions in diverse cloud providers and regions, using the methodologies of sustainable computing.

3.4 Experimental Design Scenario

The experiment is characterized by a multi-scenario approach dedicated to evaluating the performance of the framework under particular deployment and workload conditions. The experimental setup includes deployment scenarios with typical real-world use cases for multi-cloud serverless applications. Single-region multi-cloud deployment's objective is to check the framework's performance concerning resource optimization as it would be when single-function geographical constraints are considered with the ability to span several cloud providers, hence use strategic layer cloud selection decisions under regional constraints. Multi-region distributed architectures, on the other hand, involve performance evaluations by functions being deployable globally throughout different regions and providers where local geographical data and cross-region communication costs are the primary issues in placement optimization for tactical layer. Each of the experimental scenarios is applied with various workload settings, for instance, periodic batch processing jobs that are predictable in execution

---

[2] https://github.com/google/cluster-data/blob/master/ClusterData2019.md

[3] https://github.com/spcl/serverless-benchmarks

[4] https://portal.electricitymaps.com/datasets

times, bursty web traffic that is characterized by spikes of high variance and sustained streaming analytics workloads that require consistent resources to be available with a strict latency requirement.

Performance indicators collected are end-to-end request latency with the distribution of percentiles, cost metrics that include compute expenses, energy consumption, carbon footprint calculations, and service level agreement compliance rates. The efficiency of the framework's optimization is measured through a comparative analysis between the used baseline approaches, which include greedy heuristic algorithms that make decisions based on the immediate cost alone without considering the long-term implications, and the current established DRL algorithm used for serverless cost optimization as outlined in literature (Chen et al. 2025).

3.5 Evaluation Process and Metrics

An all-inclusive evaluation framework is utilized for gauging the efficacy of the proposed multi-cloud serverless orchestration framework. Core cost metrics are designed in such a way that they can be easily compared with the already existing benchmarks, including the per-one-million-invocations deployment cost. The assessment presents the cost reduction in percentage to the objective baselines, projecting the target performance as close to the optimal solution as possible. Performance metrics are conceived from baseline papers that involve end-to-end latency figures from request receipt to service completion. Service time optimization is applied in the same way as Jiang et al.'s ECOLIFE framework, which measures the temporal overhead due to different clouds and infrastructure. Acceptance metrics consist of the successfully deployed requests over the total incoming requests, which would shed light on the system's capabilities and its efficiency in resource allocation under diverse loading conditions. One major metric is decision-making time, which is the computational overhead in the DRL proposal against the baseline solvers. The framework compares against theoretical bounds in terms of resource efficiency, operational metrics, and energy usage.

4 Design Specification

The HDRL architecture for multi-cloud serverless orchestration is featured in this framework architecture, as seen in Figure 2. To achieve the desired results, the strategic decision is chosen first, followed by the tactical and operational levels, respectively. The cloud provider has the ability to allocate one of the three, AWS, Azure, or GCP, decided by a DQN agent that learns from the historical workload data.

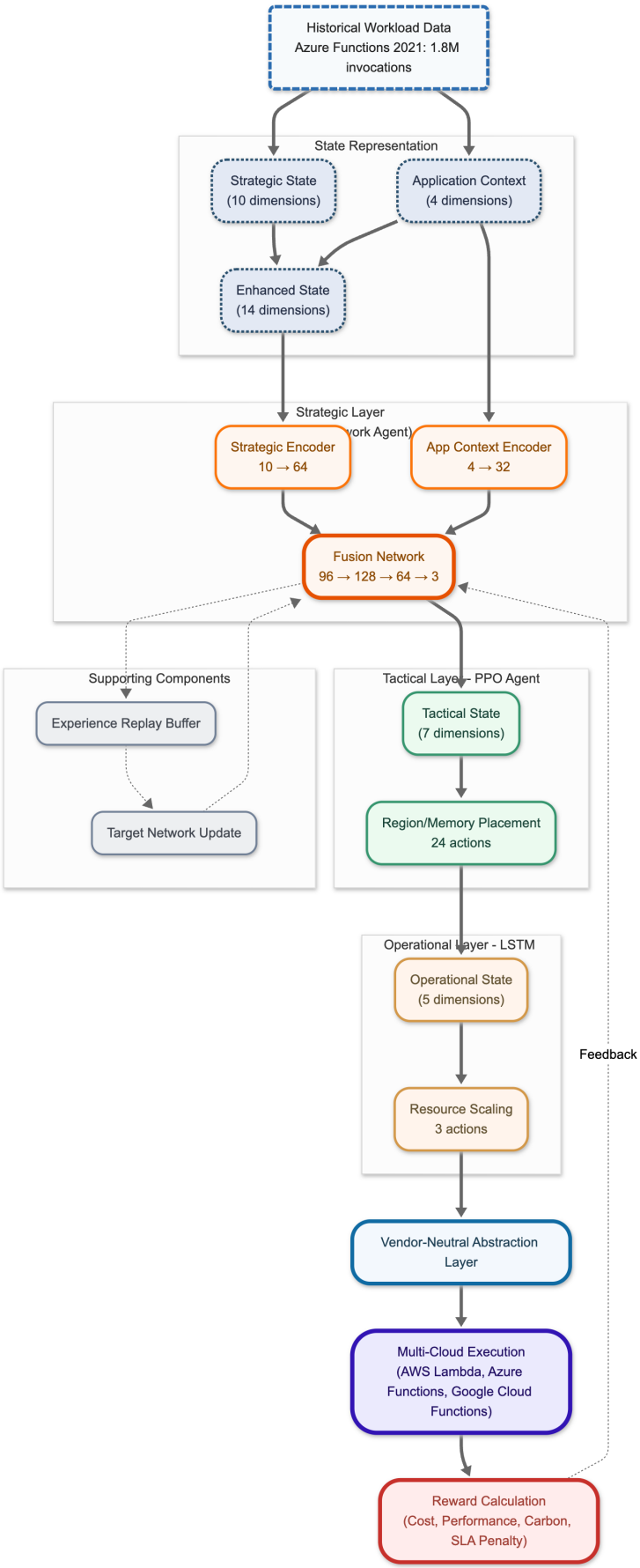
4.1 State Representation and Application-aware Learning

The strategic decision-making layer processes a fourteen-dimensional state space that combines features with application-specific context and pre-computed strategy for selection of the cloud provider. The state component of the strategy comprises ten dimensions that are pre-processed. The dimensions include time-related features like hour of the day (0-23), day of the week (0-6), weekend indicator (binary), and business hours flag (binary); workload characteristics such as invocation rate (normalized requests per second) and burstiness

indicator (binary classification); and historical performance metrics like average execution time, average cost per invocation, average carbon footprint, and allocated memory capacity. The application context that the user contributes for application insight provides the four dimensions, namely, cold start rate, SLA violation rate, normalized average invocation rate, and workload type indicator. The cold-start rate dimension represents the historical probability of initialization delays of the container, the SLA violation rate quantifies the past deadline miss frequencies, the normalized average invocation rate is scaled to the 0-1 range, while the workload type indicator is between the standard and bursty traffic patterns. The feature enables the DQN agent to alter the strategic design according to the specific application requirements; thus, it is possible to apply penalties that are differential to cold starts that adversely affect the latency consumption of the bursty workloads, whereas bonuses are paid for the SLA compliance on the high-risk applications that are maintained.

4.2 DQN Architecture

The DQN implements a dual-pathway encoder model for the DQN that performs a distinct combination of processing strategic and application features through separate specialized networks before fusion, enabling the model to learn distinct representations for environmental context and application characteristics. The strategic encoder pathway converts the ten-dimensional strategic state into a fully connected layer with 64 output nodes, ReLU activation to introduce non-linearity, LayerNorm normalization to stabilize training across variable batch sizes right after the last layer, and 10% dropout regularization to prevent overfitting on the 1,264,946 training samples. The application context encoder processes the four-dimensional application features through a parallel pathway with 32 output nodes, identical activation and normalization schemes, and 10% dropout, creating specialized embeddings that capture application-specific behavioral patterns. Afterward, the fusion layer concatenates the 64-dimensional strategic embedding derived from the strategic encoder with the 32-dimensional application embedding created from the application context encoder to form a 96-dimensional representation, which goes through a hidden layer with 128 ReLU activation and LayerNorm, a 64-node intermediate layer, a 20% dropout for regularization, and finally a 3-node output layer producing Q-value estimates for the three cloud provider actions. The initialization of Kaiming normal is applied to all linear layer weights with the assumption of ReLU nonlinearity, while bias terms are initialized with small positive values of 0.01 to help early learning, and the network has 22,179 trainable parameters, which are optimized through the Adam algorithm with a conservative learning rate of 0.0001.


Historical Workload Data
Azure Functions 2021: 1.8M invocations
State Representation
Strategic State (10 dimensions)
Application Context (4 dimensions)
Enhanced State (14 dimensions)
Strategic Layer
Strategic Encoder 10 → 64
App Context Encoder 4 → 32
Fusion Network 96 → 128 → 64 → 3
Supporting Components
Experience Replay Buffer
Target Network Update
Tactical Layer – PPO Agent
Tactical State (7 dimensions)
Region/Memory Placement 24 actions
Operational Layer - LSTM
Operational State (5 dimensions)
Resource Scaling 3 actions
Feedback
Vendor-Neutral Abstraction Layer
Multi-Cloud Execution (AWS Lambda, Azure Functions, Google Cloud Functions)
Reward Calculation (Cost, Performance, Carbon, SLA Penalty)

4.3 Action Space and Decision Framework

The strategic layer action space is made up of three discrete actions corresponding to cloud provider selection decisions: Action 0 selects AWS Lambda, action 1 chooses Microsoft Azure Functions, and action 2 designates GCP Cloud Functions as the target deployment platform. The epsilon-greedy exploration strategy supports the dual objectives of exploitation of the learned policies as well as the exploration of the alternative actions by a decaying epsilon that starts at a maximum of 1.0 for the initial exploration and goes through an exponential decay over 10,000 training steps to arrive at a minimum of 0.01 imposed to continue the exploratory action with 1% of the training period. The training results confirm that the epsilon decay is stabilized around episodes 3-4, dropping from the beginning values of 0.38 to 0.01, providing the platform for the agent to learn the optimal policies from exploratory learning to exploitation. The tactical layer is furthered by the twenty-four actions, which consist of the combinations of the geographical area and memory allocation, while the operational layer executes three actions of scaling resource adjustment by scaling up, keeping the current allocation, and scaling down depending on demand prediction.

4.4 Experience Replay and Target Network

The DQN agent has an experience replay with the circular buffer to store up to 100,000 state-transition tuples to break the temporal correlation in the sequential training data and enable the efficient re-deployment of past experiences. Each stored transition also contains the state vector (14-dimensional vector), action selected (0-2 integer), reward received (scalar), state next (14-dimensional vector), and termination (boolean), which simplifies mini-batch sampling, which is generally a prerequisite for stable gradient-based learning. The training procedure involves randomly sampling batches of 64 transitions from the replay buffer, calculating Q-values from the online network, obtaining target Q-values with a separate target network updated every 1,000 training steps, and minimizing the Smooth L1 loss (Huber loss with beta=1.0) between predicted and target values. Moreover, the aggressive gradient clipping of the 0.5 threshold is used to avert any exploding gradients, while the target value clamping is done to restrict the Q-estimates within the [-10.0, 10.0] range, which helps in maintaining numerical stability during the training. Finally, the NaN detection mechanisms are comprehensive and extend to the states, Q-values, and loss terms at each training step; thus, the anomalies are recorded, and updates are skipped where instabilities have occurred. The volatile yet definitive monitoring of NaN states, Q-values, and loss terms on each training step, not just bypassing updates for numerical instabilities but also keeping an anomaly count, attests to the reliability of the training process across the 50-episode training run through 10,000 samples per episode.

4.5 Multi-Objective Reward Function

The framework employs a multi-objective optimization tool through the weighted reward function for the promotion of environmental sustainability, economic efficiency, and operational performance objectives. Cost rewards are computed by normalizing the compute cost per invocation by dividing by a reference threshold of $1.00, thus yielding higher rewards for lower costs when they are subtracted from unity. The applied weight is alpha=0.4, referring to the strong organizational emphasis on cost optimization. Performance rewards are based on

the comparison between the invocation latency and the reference threshold of 1,000 milliseconds, with the normalized latency value being subtracted from unity to reward the faster execution time and a weight beta=0.4, which denotes the importance of cost consideration. The carbon rewards deal with the environmental implications of carbon emissions by normalizing the carbon footprint against a 100-gram CO_2 equivalent reference value, subtracting from unity to reward the fewer emissions, and applying a weight of gamma=0.2 to show the organizational commitment to sustainability as a secondary optimization criterion. Application-aware reward shaping supplements these base rewards with contextual adjustments: burst issues are penalized by -0.2 if cold starts, which elevate the latency sensitivity, are the case for bursty applications, and a +0.1 extra reward is given for the presence of SLA compliance on applications that have a historical violation rate of more than 10% to secure the reliability of high-risk workloads.

4.6 Vendor-Neutral Abstraction Layer

The abstraction layer secures a central architectural role by sectionalizing the decision-making at a high level from the implementation specifics on the part of the provider; this way it enables seamless cross-cloud function deployment without introducing vendor lock-in dependencies. The abstracting layer attaches to this as a translation mechanism, which brings together the general function deployment specifications and the platform-specific API calls that align with each of the states corresponding to the AWS Step Functions state machine definitions, Azure Durable Functions orchestration patterns, Google Cloud Workflows YAML syntax, and the OpenFaaS function specifications.

5 Implementation

You will of course want to discuss the implementation of the proposed solution. Only the final stage of the implementation should be described.

It should describe the outputs produced, e.g. transformed data, code written, models developed, questionnaires administered. The description should also include what tools and languages you used to produce the outputs. This section must not contain code listing or user manual description.

6 Evaluation

The purpose of this section is to provide a comprehensive analysis of the results and main findings of the study as well as the implications of these finding both from academic and practitioner perspective are presented. Only the most relevant results that support your research question and objectives shall be presented. Provide an in-depth and rigorous analysis of the results. Statistical tools should be used to critically evaluate and assess the experimental research outputs and levels of significance.

Use visual aids such as graphs, charts, plots and so on to show the results.

6.1 Experiment / Case Study 1

. . .

6.2 Experiment / Case Study 2

. . .

6.3 Experiment / Case Study 3

. . .

6.4 Experiment / Case Study N

. . .

6.5 Discussion

A detailed discussion of the findings from the N experiments / case studies. Note that this discussion will have a lot more detail than the discussion in the following section (Conclusion). You should criticize the experiment(s), and be honest about whether your design was good enough. Suggest any modifications and improvements that could be made to the design to improve the results. You should always put your findings into the context of the previous research that you found during your literature review

7 Conclusion and Future Work

Restate your research question, your objectives and the work done. State how successful you have been in answering the research question and achieving the objectives. Restate the key findings. Discuss the implications of your research, talk about the efficacy of your research, and discuss its limitations.

Describe any proposals for future work or potential for commercialisation. Present MEANINGFUL future work. Sweeping more parameters in your simulation / model / platform is probably not meaningful. More discuss what could a follow up research project do, to better / differently approach / extend etc. your work.

References

References should be formatted using APA or Harvard style as detailed in NCI Library Referencing Guide available at https://libguides.ncirl.ie/referencing
You can use a reference management system such as Zotero or Mendeley to cite in MS Word.

Beloglazov, A. and Buyya, R. (2015). Openstack neat: a framework for dynamic and energy-efficient consolidation of virtual machines in openstack clouds, *Concurrency and Computation: Practice and Experience* 27(5): 1310–1333.

Feng, G. and Buyya, R. (2016). Maximum revenue-oriented resource allocation in cloud, *IJGUC* 7(1): 12–21.

Gomes, D. G., Calheiros, R. N. and Tolosana-Calasanz, R. (2015). Introduction to the special issue on cloud computing: Recent developments and challenging issues, *Computers & Electrical Engineering* 42: 31–32.

Kune, R., Konugurthi, P., Agarwal, A., Rao, C. R. and Buyya, R. (2016). The anatomy of big data computing, *Softw., Pract. Exper.* 46(1): 79–105.